

Secure development with Copilot Chat in VS Code

A coding agent can read your files, change them and run commands on your laptop. That makes it powerful, and it means **you** are responsible for what it does. This guide covers the habits that keep you safe. Keep it open while you work.

1. Know what each mode can do

The dropdown at the bottom of the chat decides how much power you hand over.

Mode	Can read files	Can change files	Can run commands	Use it for
Ask	Only what you attach	No	No	Questions, explanations
Plan	Yes	No	No	Working out <i>what</i> to do before doing it
Agent	Yes	Yes	Yes, after you approve	Building and fixing

Rule of thumb: use the least powerful mode that gets the job done. Plan first, then switch to Agent to build.

2. Know what leaves your laptop

Everything the agent looks at is sent to the AI model in the cloud: your prompt, the files it reads or you attach, selected code, and the output of any commands it runs.

- **Keep secrets out of the workspace.** Passwords, API keys and tokens in files like `.env` can be read by the agent and end up in the chat. Store real secrets in a password manager or a secrets vault, not in the project folder.
- **Never use real personal data.** Don't paste real customer, employee or source data into the chat, and don't keep it in the project. Use made-up test data, like the sample tips in this repo.
- **Ask your IT admin** what your organisation's Copilot plan allows. Admins can exclude files from Copilot and turn features on or off, but check exactly what those rules cover before you rely on them.

3. Read before you approve

In Agent mode, Copilot asks before running a terminal command. **That approval is your main safety control.**

- Read the command. If you don't understand it, decline it and ask "*What would that command do?*"
- Be extra careful with anything that **deletes** (`rm`, `Remove-Item`, `del`), **undoes history** (`git reset`, `git clean`), **installs** (`pip install`, `npm install`) or **talks to the internet** (`curl`, `Invoke-WebRequest`).
- **Don't turn on "auto-approve everything"**, even when the prompts feel slow. You can let harmless commands through automatically and make risky ones always ask. See `chat.tools.terminal.autoApprove` in this repo's `.vscode/settings.json` for an example.

4. Review every change, and keep a save point

- **Commit before you let the agent work.** Then `git diff` shows exactly what it changed, and `git restore .` undoes it all. Git is your safety net, not the agent's *Undo* button.
- **Review the changed files** before you click **Keep**. Look out for changes to files you didn't ask about, especially settings (`.vscode/`, `.github/`), tests, and `requirements.txt`.
- **Work in small steps.** One feature per prompt. A big, messy change is hard to check.
- **Check its claims.** "All tests pass" means nothing until you have run the tests yourself.

5. Treat what the agent reads as untrusted

The agent follows instructions, and it can't always tell *your* instructions apart from text it reads in files, web pages or tool results. This is called **prompt injection**.

- **Only trust folders you know.** When VS Code asks "*Do you trust the authors?*", say yes only for code from a source you trust. A project can include settings and instruction files (`AGENTS.md`, `.github/copilot-instructions.md`) that change how the agent behaves.
- **Be careful with content from outside:** web pages, downloaded files, issue text, emails. Any of it could contain hidden instructions like "*ignore previous instructions and...*".
- **Instruction files guide the agent, but they can't force it.** A rule in `AGENTS.md` like "don't read `.env`" is a request. Only settings, permissions and access rights actually block anything.

6. Be picky about what you add

- **Packages:** when the agent wants to add a library, ask why it's needed. Check that the name is spelled correctly and that it is a well-known package. Attackers publish look-alike packages hoping someone installs them by mistake.
- **Extensions and MCP servers** give the agent new tools, such as access to databases, browsers or cloud accounts. Only install ones from sources you trust, and switch off tools you don't need (the *Configure Tools* button in the chat).

7. Ask the agent to check its own work

The agent is good at finding problems when you ask it to look for them:

Review the changes you just made for security problems. Think about malicious input, leaked secrets or personal data, and anything that could be abused. Don't fix anything yet, just list the problems.

Then fix them one at a time, **each with a test**.

Checklist

- ☐ Right mode for the task (Plan before Agent)
- ☐ No real secrets or personal data in the workspace or the chat
- ☐ Work committed before the agent starts
- ☐ Every terminal command read before approving
- ☐ Every change reviewed with `git diff` before committing
- ☐ New packages, extensions and tools checked before adding
- ☐ Outside content treated as untrusted
- ☐ Tests run by you, not just reported by the agent